

# **Constraint-Based Resource Sharing Problem (NP-Hard Problem)**

## **Introduction:**

Resource sharing is an important problem in computing systems, project management, and network environments. The objective is to allocate limited resources among competing tasks while satisfying fairness and priority constraints.

## **Problem Description:**

A set of tasks requires access to a limited set of shared resources. The allocation process must ensure that constraints such as maximum usage limits, fairness, and task priorities are respected.

## **Constraints:**

- Resources have limited availability.
- No resource can exceed its maximum usage limit.
- Fairness must be maintained among tasks.
- Higher-priority tasks may receive preference during allocation.

## **Solution Approach:**

A backtracking-based solution is employed to assign resources to tasks. During the allocation process, constraint checks are performed. If an assignment violates any constraint, that branch is pruned, reducing unnecessary exploration.

## **Analysis:**

Backtracking guarantees that all feasible allocations are explored. Pruning techniques improve efficiency by eliminating invalid assignments early. Despite these improvements, the number of possible allocations grows exponentially with the number of tasks and resources.

## **Correctness:**

The algorithm checks every valid allocation possibility while rejecting those that violate constraints. Therefore, any allocation produced by the algorithm satisfies all specified requirements.

## **Complexity:**

- Time Complexity:  $O(r^n)$
- Space Complexity:  $O(n)$

Where:

- $r$  = Number of resources
- $n$  = Number of tasks

**NP-Hard Classification:**

The Constraint-Based Resource Sharing Problem is NP-Hard because it generalizes many difficult combinatorial optimization problems. Finding an optimal allocation often requires examining a large number of possible assignments.

**Conclusion:**

The Constraint-Based Resource Sharing Problem illustrates the complexity of resource allocation under constraints. Backtracking with pruning provides a systematic solution, but computational complexity remains high for large-scale instances, reflecting its NP-Hard nature.